

Secure EHR Insight & Clinical Validator : Chen-#1

prerequisite: python, Database, RAG, OOP, llm, workflow, Agent

Hospitals $\rightarrow$ EHR

(Electronic Health record) $\rightarrow$ admission $\rightarrow$ relational data
 $\rightarrow$ prescription
 $\rightarrow$ lab results
 ↓
 millions of row

problem statement: I'm to let their doctors ask question in NLP about patient histories to save time.

Data $\rightarrow$ RAG, Text2SQL, Vector DB $\xrightarrow{\text{execute}}$ llm

hospital → ILM
(raw patient data) ↓ violation HIPAA → HIPAA ←

Solution: —

- privacy first pipeline
- standards
- personal data

AI-SDLC

project



→ coding

→ QA → write test

→ code review

→ vibe coding on steroids

→ Developer → plan
steps .md
task
edge

→ boiler plate → plan.md → structure
↳ execution → system design

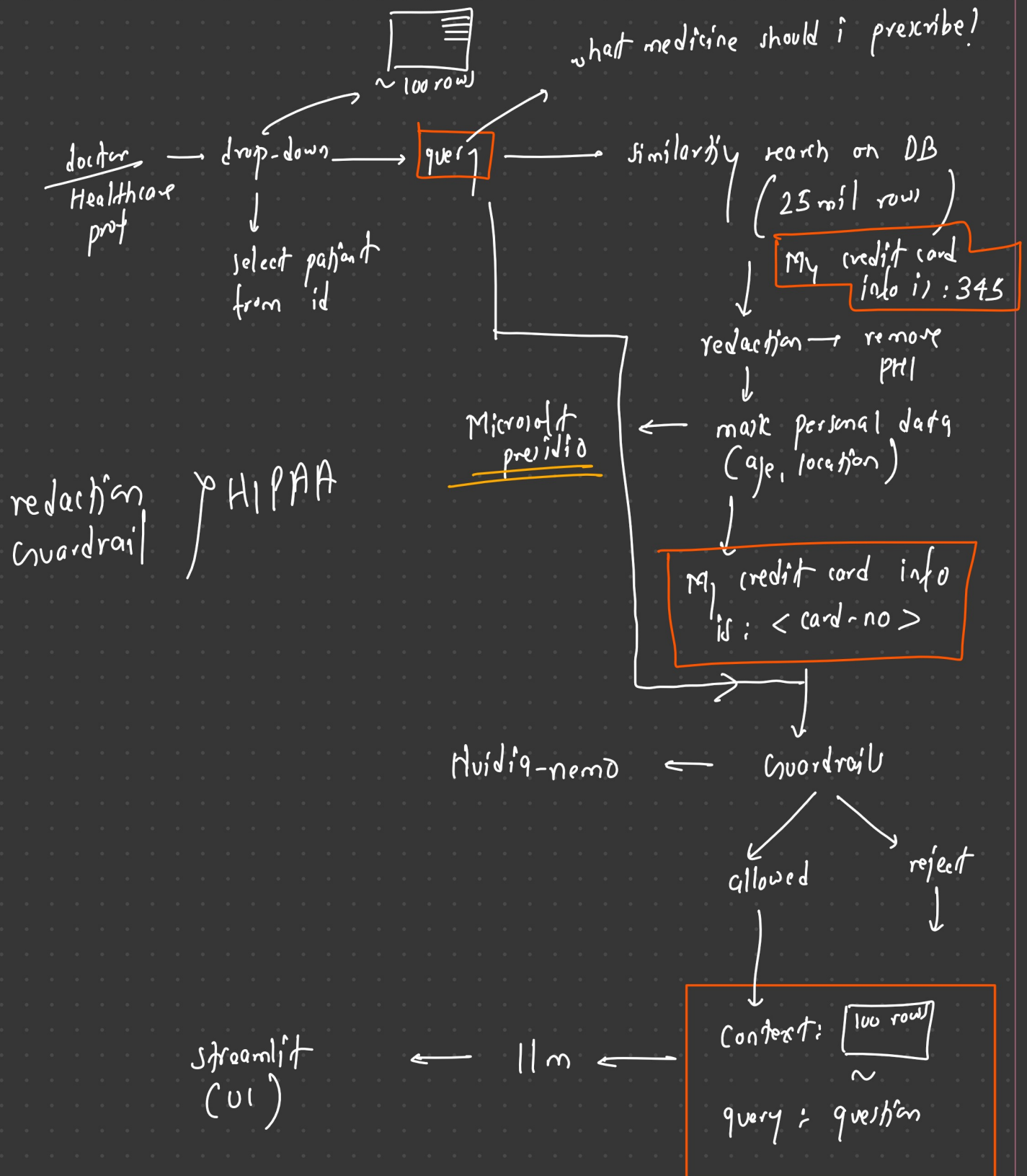
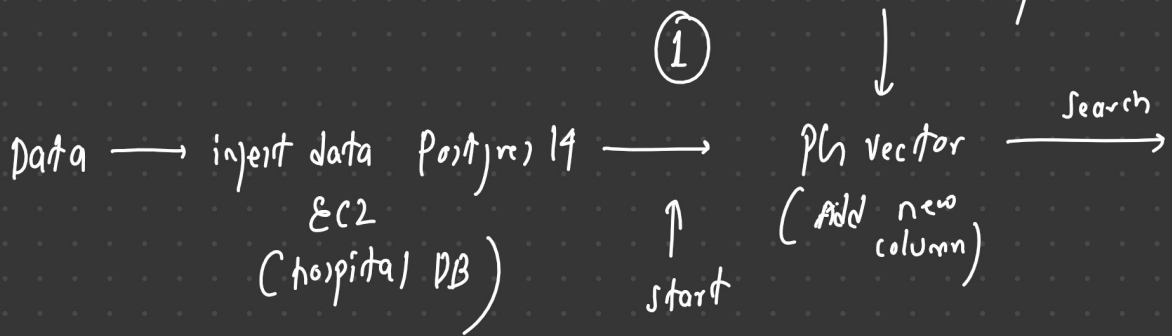
→ AI, unit, Testcase, report
↓ review

→ AI → PR
→ security (security flaw)
→ performance regression

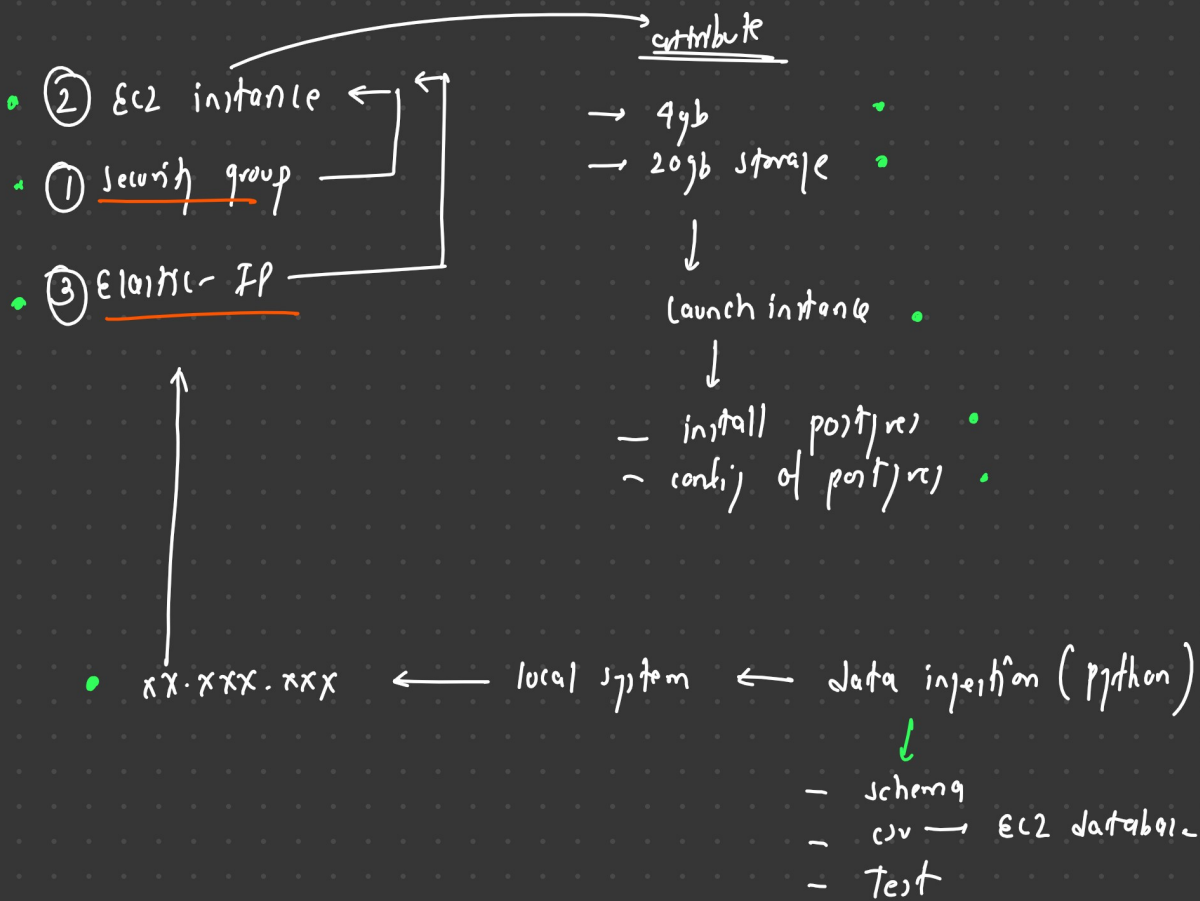
Maintain
update

→ hotfixes, update the doc

In-depth - Overview



phase-0



uv → global env

uv venv

folder
└ env

Data is injected

phase-1

- Install pg vector

Database

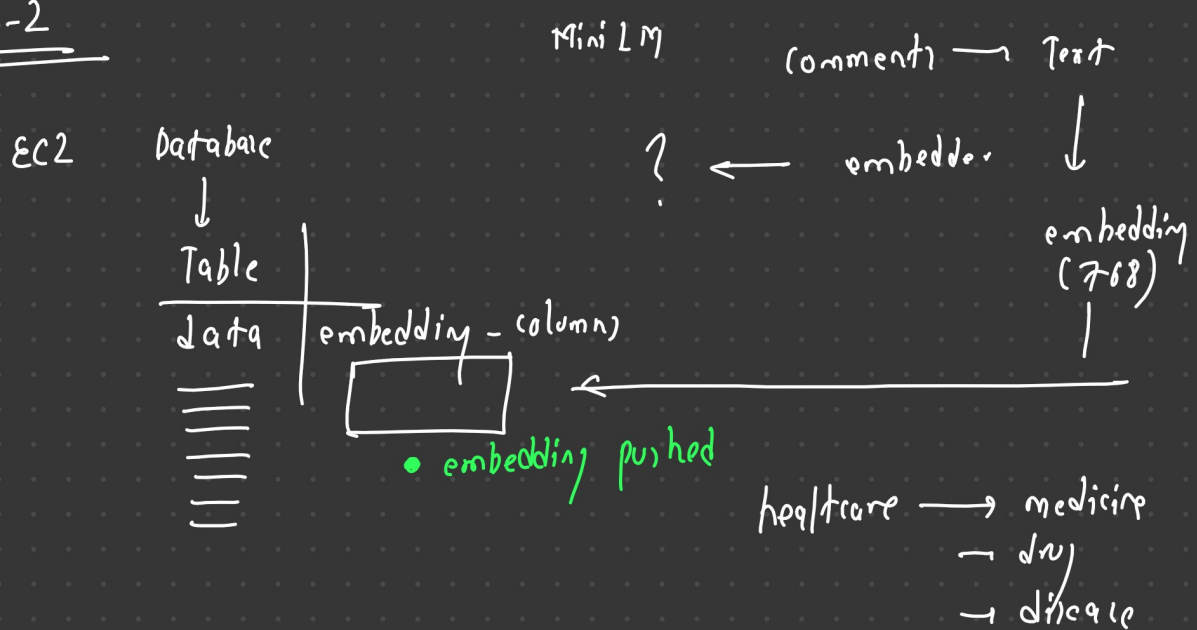
Vector

patient_encounter

clinical-embedding vector (768)

lm
↓
embedding

phase - 2



NULL → Clinical-embedding
↓
not null anymore

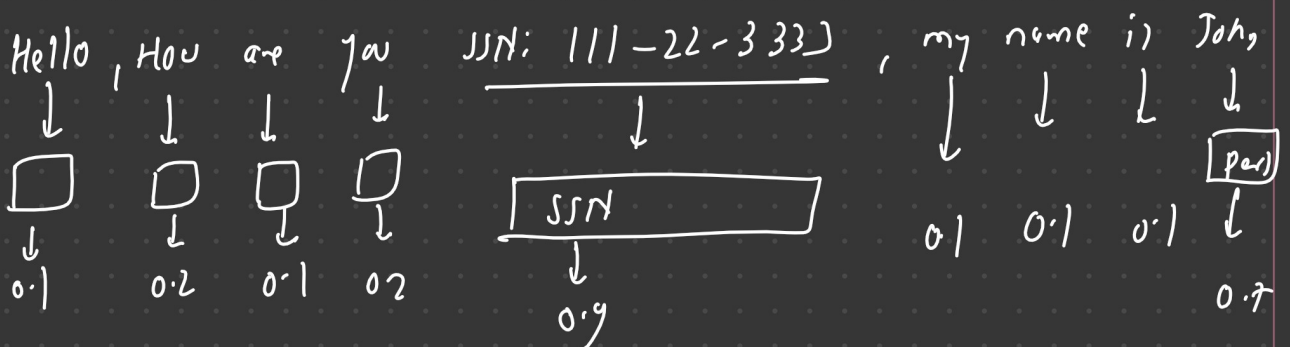
→ pushing embedding takes time

→ EC2, postgres, Database, Table, injected-data, added vector extension to db, create a new embedding column, created embedding from 'combination of patient details', pushed all embedding to database. ——— x ——— demo

→ EC2 → 11,000 rows of embedding (20 min)

→ ↓
selected this in .env

query → embedding $\xrightarrow[\text{cosine similarity}]{\text{search}}$ postgres



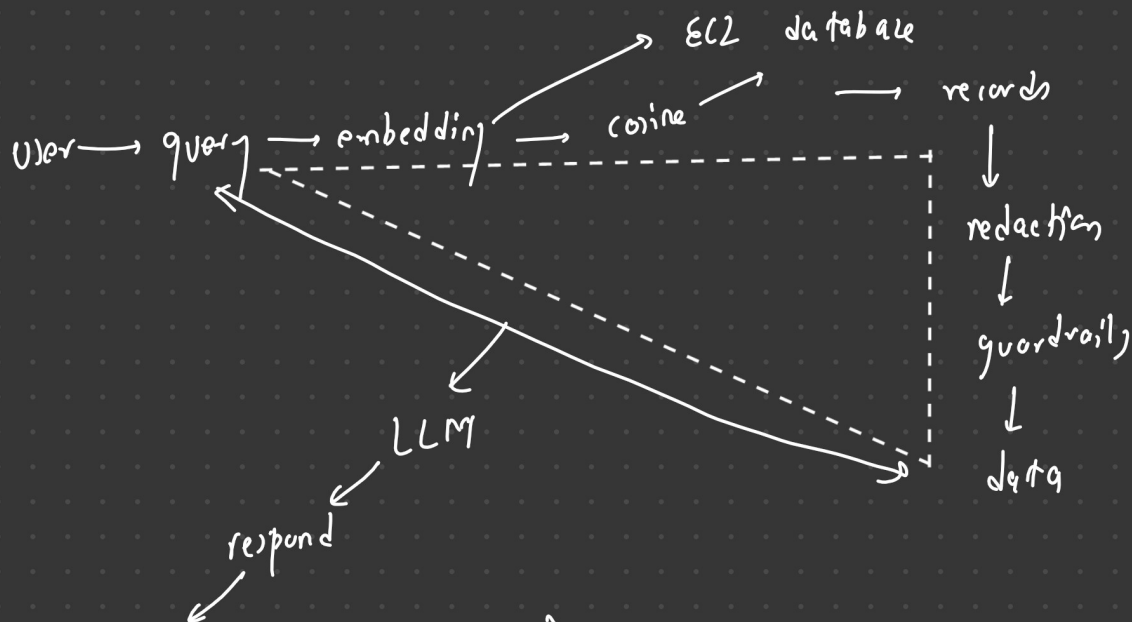
6.43 > redaction

Next-phase

guardrails → .co → create → flow

LLM → reject
↓
→ accept → no error

API → Deepseek API → cheap, powerful



(trial-api)
api-1 → test(query) → response
(query) →

api-2 → chat → query
↓
↓
↓
yet → etc →

api-3 → patient
get all patient-id



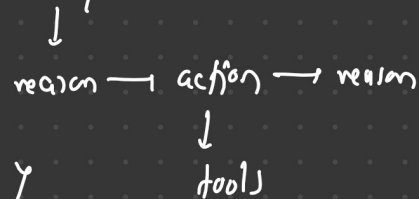
user: m)
bot: m)



prompt = { clinical_context { dbp: user: () }
↓
gpt4oai



langgraph → Agent → Task simple → react agent



workflow = { { role: user : que: [] },
{ role: agent: respen: [] },

conversation history → llm api call

chat = [] → role: user, content: Hello

user: Hello

